Logan Griffin

EN.605.613.81.FA25

Introduction to Robots

Final Project

12/7/2025

How to run the code:

- ROS Version: jazzy
- Docker image: tiryoh/ros2-desktop-vnc:jazzy
- To run the code:
    1. Start the docker container and open the image
    2. open a new terminal
    3. navigate to the directory holding the src folder
    4. run “colcon build” to build the gazebo_controller package
    5. run “source install/setup.bash” to update environment variables
    6. run “ros2 launch gazebo_controller full_simulation.launch.py”. This will launch Rviz and Gazebo. To run alternate mazes, use the original prompt and append any of the following: maze:=Maze_hr, maze:=Maze_ng, maze:=Maze_ql_1. For example, to launch the simulation with maze_hr, run “ros2 launch gazebo_controller full_simulation.launch.py maze:=Maze_hr”
- NOTE: any changes that are made to the code require a rebuild (“colcon build”) and a source (“source install/setup.bash”)
- For additional information, see final_template_jazzy/src/gazebo_controller/README.md

Approach:

The robot navigates the maze by running a version of EKF SLAM in combination with A* search.

Everything starts with the mapper node, as it is responsible for publishing the occupancy grid and estimated robot poses. The mapper node subscribes to odometry messages that provide the linear and angular velocities of the robot based on wheel rotation rates. Those values are then integrated and used as the motion model prediction for the EKF. The mapper node also subscribes to messages from the IMU sensor on the robot. The orientation from the IMU messages is used as the measurement to update the EKF. The mapper node uses subscription callback methods on the messages to constantly update the estimated state of the robot using the filter. Lidar scans are received by the mapper node as well. Once a new scan comes in, the mapper uses the stored, estimated pose from the EKF to build an occupancy grid in the maze world frame. The occupancy

grid is initialized to unknown when the maze is created and is aligned with the maze world frame. The size and position of the initial grid is based on the resolution and origin designated in the maze configuration file. There are two additional helper arrays: one to store hits in cells, and one to store misses. The mapper takes the ranges and angles from the lidar scan and marches them starting at the robot's estimated pose. Each grid cell location along the ray is incremented in the misses array. At the end of the ray, the grid cell location is incremented in the hits array. The actual method to build the occupancy grid grabs the number of hits and misses from the helper arrays and uses simple probability to determine whether that cell should be marked as occupied, free, or unknown. A square buffer is then applied to all the cells that were marked as occupied in the grid to inflate their occupation into free space. The size of the buffer is tunable for different map characteristics, and is based on the resolution of the map. The idea is that the inflated obstacles will allow the robot to achieve sufficient clearance around corners and maintain a certain distance from the obstacle walls. Finally, the occupancy grid and estimated pose are published and received by the planner node.

The planner node is responsible for feeding goal poses to the controller node. When the simulation is launched, the planner node subscribes to the message that creates the goal locations in the map. It stores the goal locations in a dictionary for later use in path planning. The planner node runs on a timer, but the stored occupancy grid and estimated pose are constantly updated by subscription callback methods. Once the timer goes off, the planner takes the estimated robot state and calculates which of the maze goals is closest and makes it the target. Next, the planner runs an A* search from the estimated robot coordinates to the target goal coordinates. If no path is found, the A* search will simply return whatever path was generated previously. This ensures that the robot keeps moving even if it enters a region in the buffer that might appear to be occupied. The robot pose and goal location are transformed into grid coordinates to keep everything consistent. Finally, the planner node takes the generated A* path, advances along it a certain distance, transforms that grid cell into maze world coordinates, and publishes it to the goal pose topic. The advancing parameter can be tuned for different mazes as well. A higher value will smooth the robot motion, but it can also cause navigation issues like cutting corners.

The controller node receives the goal pose and transforms it into the odom frame. On a timer, the controller will lookup the transform from odom to the vehicle_blue truth pose and then calculate the PID error from the received goal to generate velocity commands. The last step in the navigation loop is for the controller to publish the command velocities, which are received by the simulator to drive the robot.